

Algorithmique avancée avec Scratch — préparation à l'épreuve de programmation du brevet

Objectifs de la leçon

- ✓ Réviser et combiner boucles, variables et conditions dans un même programme
- ✓ Utiliser des conditions imbriquées (**si...alors** à l'intérieur d'un autre **si...alors**)
- ✓ Créer et manipuler une liste (ajouter, retirer, lire un élément)
- ✓ Créer et utiliser un bloc personnalisé pour réutiliser une suite d'instructions
- ✓ Lire un algorithme donné et prévoir son résultat sans l'exécuter
- ✓ Compléter un algorithme à trous pour qu'il produise un résultat attendu
- ✓ Écrire un algorithme simple à partir d'un énoncé, en pseudo-code ou en blocs Scratch

L'essentiel à retenir

- 1 L'épreuve pratique de programmation du brevet (DNB) évalue la capacité à comprendre, compléter et écrire de courts algorithmes, le plus souvent sous forme de blocs façon Scratch (ou en pseudo-code proche du langage naturel). Trois types d'exercices reviennent systématiquement : lire un algorithme et prévoir ce qu'il affiche ou calcule ; compléter un algorithme auquel il manque une ou plusieurs instructions pour qu'il fonctionne correctement ; écrire un petit algorithme complet à partir d'un énoncé (par exemple, calculer une moyenne, tester une condition, ou répéter une action un certain nombre de fois).
- 2 Révision des boucles : la boucle **répéter n fois** exécute un bloc d'instructions un nombre de fois fixé à l'avance (utile quand on connaît le nombre de répétitions), tandis que la boucle **répéter jusqu'à...** (ou **répéter tant que...**) exécute les instructions tant qu'une condition n'est pas encore vraie (ou reste vraie), utile quand on ne connaît pas à l'avance le nombre de répétitions. Exemple : pour afficher les nombres de 1 à 10, on utilise une variable **compteur** initialisée à 1, une boucle **répéter jusqu'à ce que compteur > 10** qui affiche le compteur puis l'incrémente de 1 à chaque tour.
- 3 Les conditions imbriquées consistent à placer un bloc **si...alors** à l'intérieur d'un autre bloc **si...alors**, pour tester plusieurs critères successifs. Exemple : un programme qui affiche une appréciation selon une note sur 20 peut tester d'abord **si note ≥ 10 alors** (réussite), et à l'intérieur, un second test **si note ≥ 16 alors afficher Très bien, sinon afficher Bien** ; à l'extérieur, le **sinon** du premier test affiche **Insuffisant**. Cette imbrication permet de distinguer plusieurs cas de figure avec précision.
- 4 Une liste est une structure qui permet de stocker plusieurs valeurs sous un même nom, contrairement à une variable qui ne stocke qu'une seule valeur à la fois. Sur Scratch, on crée une liste dans la catégorie **Variables**, puis on utilise les blocs **ajouter [valeur] à [liste]** pour ajouter un élément à la fin, **supprimer [position] de [liste]** pour retirer un élément, et **élément [position] de [liste]** pour lire la valeur stockée à une position donnée. Exemple : pour calculer la moyenne de plusieurs notes saisies par l'utilisateur, on les stocke dans une liste **notes**, puis on parcourt la liste avec une boucle pour additionner toutes les valeurs avant de diviser par le nombre d'éléments.
- 5 Un bloc personnalisé (ou **ma brique**) permet de créer sa propre instruction réutilisable, regroupant plusieurs blocs déjà utilisés ensemble. On le crée dans la catégorie **Mes blocs**, en lui donnant un nom (par exemple **dire bonjour** ou **calculer carré**), éventuellement avec des paramètres (des valeurs que l'on peut faire varier à chaque appel). Cela évite de dupliquer plusieurs fois la même suite d'instructions dans un programme, et rend le programme plus lisible et plus facile à corriger.

Algorithmique avancée avec Scratch — préparation à l'épreuve de programmation du brevet

- 6 Exemple de bloc personnalisé avec paramètre : on crée un bloc **avancer_carré (côté)** qui fait avancer le lutin de **côté** pas puis tourner de 90°, répété 4 fois : cela dessine un carré de la taille voulue. On peut ensuite appeler **avancer_carré 50** puis **avancer_carré 100** pour dessiner deux carrés de tailles différentes, sans réécrire les instructions à chaque fois.
- 7 Méthode pour **lire un algorithme et prévoir son résultat** (type d'exercice fréquent au brevet) : on exécute mentalement le programme ligne par ligne, en notant sur un brouillon la valeur de chaque variable après chaque instruction, comme si l'on remplissait un tableau de suivi. Exemple : soit l'algorithme **x prend la valeur 3 ; répéter 3 fois : x prend la valeur x×2** ; on suit x : après le 1er tour x=6, après le 2e tour x=12, après le 3e tour x=24 ; à la fin, x vaut 24.
- 8 Méthode pour **compléter un algorithme à trous** : on identifie d'abord ce que le programme doit produire (donné dans l'énoncé), puis on repère les blocs manquants en analysant la logique globale (est-ce qu'il manque une initialisation de variable ? une condition ? une incrémentation dans une boucle ?). Exemple typique au brevet : un algorithme censé compter le nombre de nombres pairs dans une liste, où il manque le test **si élément est pair** (par exemple avec le reste de la division euclidienne par 2 égal à 0) — il faut alors reconnaître et compléter ce test.
- 9 Méthode pour **écrire un algorithme à partir d'un énoncé** : on décompose le problème en étapes simples avant de traduire en blocs ou en pseudo-code : quelles variables faut-il créer et initialiser ? quelle boucle est nécessaire (répéter n fois, ou jusqu'à une condition) ? quelles conditions faut-il tester ? Exemple : pour écrire un algorithme qui demande un nombre à l'utilisateur et affiche s'il est pair ou impair, on prévoit : une variable **nombre** (saisie par l'utilisateur), un test **si nombre modulo 2 = 0 alors afficher pair, sinon afficher impair**.

★ **Astuce** — relis cette fiche juste avant le cours suivant pour bien ancrer ce que tu as appris !